

Encapsulation in Java

Encapsulation is one of the four pillars of Object-Oriented Programming (OOP), along with **Abstraction**, **Inheritance**, and **Polymorphism**. It is the process of bundling data (variables) and methods (functions) that operate on the data into a single unit, typically a class. Encapsulation helps protect the data from unauthorized access and provides controlled access through methods.

Key Concepts of Encapsulation

1. Data Hiding:

- Restricting direct access to the internal state of an object by declaring variables as `private`.
- Ensures that data cannot be accessed directly from outside the class.

2. Controlled Access:

- Providing `public` methods (getters and setters) to access and update the private variables.
- Allows validation, logging, or other operations before accessing or modifying data.

3. Class as a Unit:

- Combining variables and methods into a single class to promote modularity and reusability.
-

Benefits of Encapsulation

1. **Data Security:** Protects data from accidental or malicious modifications.
 2. **Controlled Access:** Offers fine-grained control over how data is accessed or modified.
 3. **Ease of Maintenance:** Changes to the internal implementation do not affect external code, as long as the public interface (methods) remains the same.
 4. **Improved Code Readability and Flexibility:** Allows focus on "what an object does" rather than "how it does it."
 5. **Reusability:** Encapsulated classes are modular, making them easy to reuse in other projects.
-

How to Implement Encapsulation in Java

1. Declare the class variables as `private`.
 2. Provide `public` getter methods to retrieve the variable values.
 3. Provide `public` setter methods to update the variable values.
-

Example of Encapsulation

```
// EncapsulationExample.java
public class EncapsulationExample {
    // Private data members
    private String name;
    private int age;

    // Public getter for 'name'
    public String getName() {
        return name;
    }

    // Public setter for 'name'
    public void setName(String name) {
        this.name = name;
    }

    // Public getter for 'age'
    public int getAge() {
        return age;
    }

    // Public setter for 'age'
    public void setAge(int age) {
        // Adding validation to ensure age is positive
        if (age > 0) {
            this.age = age;
        } else {
            System.out.println("Invalid age. Age must be positive.");
        }
    }

    public static void main(String[] args) {
        // Creating an object of EncapsulationExample
        EncapsulationExample obj = new EncapsulationExample();

        // Setting values using setters
        obj.setName("John");
        obj.setAge(25);

        // Getting values using getters
```

```
System.out.println("Name: " + obj.getName());  
System.out.println("Age: " + obj.getAge());  
  
// Trying to set invalid age  
obj.setAge(-5);  
}  
}
```

Output:

```
Name: John  
Age: 25  
Invalid age. Age must be positive.
```
